

TheKnife

Manuale Tecnico

Progettazione e sviluppo della piattaforma

Daniele Montefiore — Matricola 736906 — Sede VA
Laboratorio Interdisciplinare B — a.a. 2024/2025
Università degli Studi dell'Insubria

Indice

1. Introduzione e obiettivi
2. Architettura del sistema
3. Protocollo di comunicazione e gestione della sessione
4. Gestione della concorrenza
5. Progettazione del software (UML e design pattern)
6. Progettazione della base di dati
 - 6.1 Analisi dei requisiti ristrutturata
 - 6.2 Schema ER concettuale
 - 6.3 Ristrutturazione dello schema ER
 - 6.4 Traduzione nello schema relazionale
 - 6.5 Script SQL e query a supporto dei servizi
7. Algoritmi e strutture dati
8. Sicurezza
9. Controllo degli errori e gestione delle eccezioni
10. Compilazione, packaging e struttura della consegna

1. Introduzione e obiettivi

TheKnife è una piattaforma client/server per poter gestire al meglio il grado di soddisfazione e comunicazione del cliente per quanto riguarda il ristorante. Il cliente ha la possibilità di esporre la propria opinione e recensire i ristoranti e trovarne dei nuovi. Il sistema è diviso in due parti principali:

- **serverTK**: il server, che comunica con PostgreSQL via JDBC e serve più client in parallelo;
- **clientTK**: il client, cioè permette l'utilizzo dell'applicazione da parte degli utenti in modo semplice..

I requisiti tecnologici delle specifiche sono: linguaggio Java 21 (multiplatforma), package `theknife`, interfaccia grafica, PostgreSQL + JDBC, build Maven con due jar eseguibili e javadoc.

2. Architettura del sistema

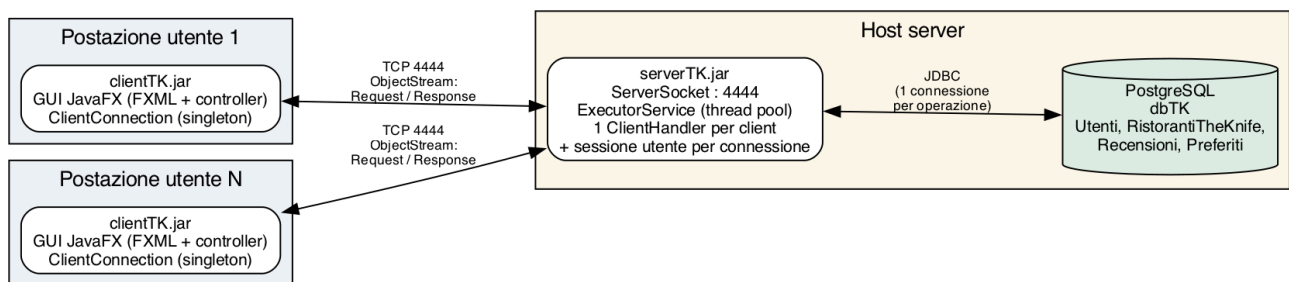


Figura 1 — Architettura distribuita della piattaforma

L'architettura è **client/server a tre livelli**: GUI (Client), logica applicativa e accesso ai dati (Server) e persistenza (PostgreSQL). Tutte le fasi di controllo, autenticazione, validazioni e chiamate al database — avvengono **sul server**: il client è il punto d'accesso che invia richieste e mostra risposte.

- Il server apre una `ServerSocket` sulla porta **4444** e all'avvio richiede host e credenziali del database.
- Ogni client accettato è servito da un `ClientHandler` dedicato, eseguito su un thread di un `ExecutorService` (cached thread pool).
- Il client mantiene una singola connessione TCP persistente (`ClientConnection`, singleton) per tutta la sessione.

3. Protocollo di comunicazione e gestione della sessione

La comunicazione usa la **serializzazione Java** su `ObjectInputStream` / `ObjectOutputStream`. Il protocollo è sincrono a scambio singolo: il client invia una `Request` e attende la `Response` corrispondente.

Classe	Campi	Costruzione e uso
<code>Request</code>	<code>operazione</code> (enum); <code>parametri</code> (<code>Map<String, Object></code>)	<code>new Request(op).con(chiave, valore)...</code> , concatenabile; il server legge con <code>get(chiave)</code>
<code>Response</code>	<code>successo</code> (boolean); <code>messaggio</code> (solo errore); <code>dati</code> (solo successo)	costruttore privato: si ottiene solo con <code>Response.ok(dati)</code> / <code>Response.errore(msg)</code>

Le classi del protocollo e i modelli (`Utente`, `Ristorante`, `Recensione`, enum `Ruolo` e `Operazione`) risiedono nel package `theknife.common`, condiviso dai due eseguibili: così client e server usano le stesse classi, così da impedire l'asincronia. Dopo ogni invio viene chiamato `ObjectOutputStream.reset()` per evitare che la cache dello stream restituisca oggetti obsoleti.

Sessione lato server. Ogni `ClientHandler` memorizza l'utente autenticato della propria connessione (`utenteLoggato`). Le operazioni riservate usano l'identità verificata dal server — mai un id inviato dal client — e verificano il ruolo all'inizio con `isCliente()` / `isGestore()`.

4. Gestione della concorrenza

Il server può servire più utenti in parallelo; gli accessi concorrenti alle risorse condivise sono gestiti su tre livelli.

4.1 Concorrenza tra client (server)

- **Un thread per client:** `Executors.newCachedThreadPool()` assegna un thread a ogni connessione, così il server accetta nuovi client senza aspettare che gli altri finiscano.
- **Nessuno stato condiviso tra client:** ogni `ClientHandler` ha i propri DAO e la propria sessione, quindi non servono lock.
- **Una connessione JDBC per operazione:** aperta e chiusa con try-with-resources (`DBConnection.nuovaConnessione()`), mai condivisa tra thread.

4.2 Concorrenza sui dati (DBMS)

I conflitti tra utenti simultanei sono risolti delegando al DBMS, che serializza le scritture in modo atomico:

- *Una recensione per utente per ristorante*: vincolo `UNIQUE (id_utente, id_ristorante)`; il tentativo duplicato genera l'errore `SQLState 23505`, tradotto in messaggio per l'utente.
- *Al massimo una risposta per recensione*: l'`UPDATE` della risposta è condizionato (`WHERE ... AND risposta IS NULL AND r.id_gestore = ?`). Se due gestori rispondessero in parallelo, solo il primo `UPDATE` modificherebbe la riga: il secondo restituisce 0 righe e viene rifiutato. Controllo e scrittura avvengono nella stessa istruzione atomica.
- *Preferito duplicato*: `INSERT ... ON CONFLICT DO NOTHING` sulla chiave primaria composta, idempotente anche con doppi click simultanei.
- *Un solo gestore per ristorante*: la presa in carico di un ristorante privo di titolare avviene con `UPDATE ... SET id_gestore = ? WHERE id = ? AND id_gestore IS NULL` . Come per la risposta alle recensioni, verifica e scrittura sono nella stessa istruzione atomica: fra più gestori che tentassero l'operazione in parallelo, uno solo gestore può modificare la riga e gli altri non ottengono alcun risultato.
- Le statistiche (media stelle, numero recensioni) non sono memorizzate ma **calcolate a run-time** con `AVG / COUNT` : mantenendo sempre i dati coerenti.

4.3 Concorrenza sul client

`ClientConnection.invia()` è `synchronized` : da un client parte una richiesta alla volta, e la risposta arriva sempre alla sua richiesta. I singleton (`DBConnection.inizializza/getIstanza`) usano metodi sincronizzati per la visibilità tra thread.

5. Progettazione del software (UML e design pattern)

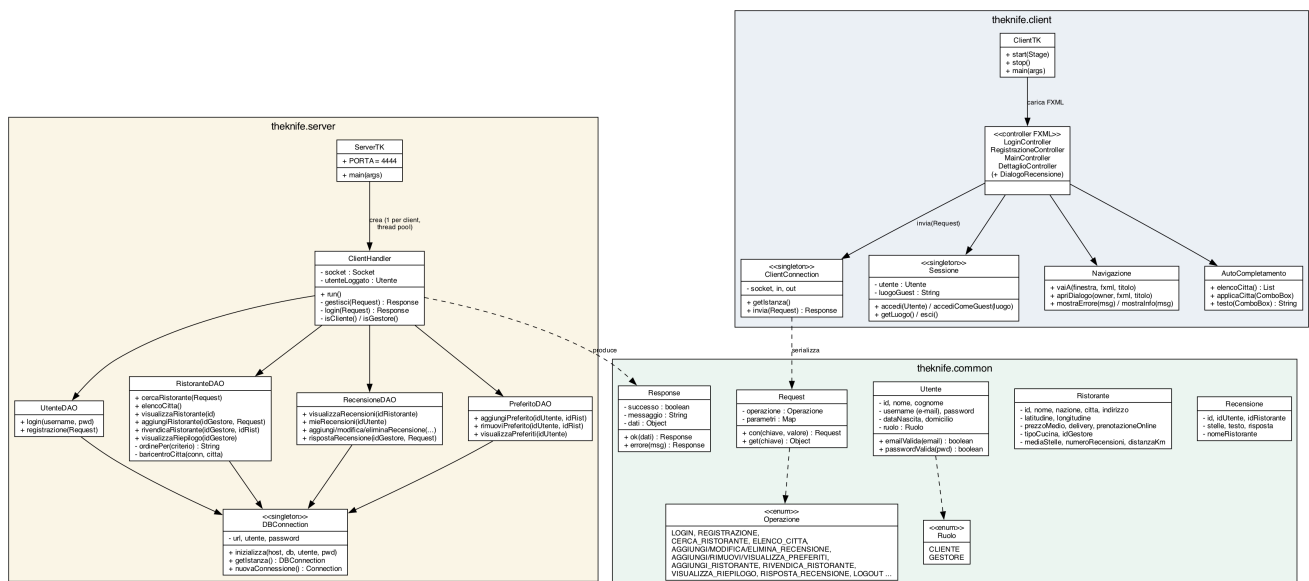


Figura 2 — Diagramma UML delle classi

5.1 Organizzazione dei package

Package	Responsabilità
theknife.common	Utente, Ristorante, Recensione (modelli), Request / Response (protocollo): condivisi da client e server.
theknife.server	ServerTK (start e accept), ClientHandler (dispatch e sessione), DBConnection (parametri e connessioni JDBC).
theknife.server.dao	Accesso ai dati: UtenteDAO, RistoranteDAO, RecensioneDAO, PreferitoDAO.
theknife.client	ClientTK / Launcher (avvio JavaFX), ClientConnection, Sessione, Navigazione, AutoCompletamento.
theknife.client.controller	LoginController, RegistrazioneController, MainController, DettaglioController, DialogoRecensione.

5.2 Design pattern adottati

Pattern	Dove	Motivazione
---------	------	-------------

DAO	<code>theknife.server.dao</code>	L'SQL è confinato nei DAO, uno per tabella: il resto del server non contiene query, e una modifica a una query non richiede modifiche in altri punti.
Singleton	<code>DBConnection</code> , <code>ClientConnection</code> , <code>Sessione</code>	Un solo punto di configurazione del DB, una sola connessione TCP per client, un solo stato di sessione della GUI.
MVC – Model-View- Controller	FXML (view) + controller + modelli common	Separazione fra layout dichiarativo, gestione eventi e dati; permette di modificare le view senza intaccare la logica applicativa.
Factory method	<code>Response.ok</code> / <code>Response.errorre</code>	Costruzione controllata e leggibile delle risposte.
Command (variante)	<code>Request</code> + enum <code>Operazione</code>	Ogni richiesta incapsula un'operazione con i suoi parametri; il dispatch è un unico <code>switch</code> sull'enum, con i controlli di ruolo (<code>isCliente()</code> / <code>isGestore()</code>) all'inizio di ogni caso riservato.

5.3 Interfaccia grafica

L'interfaccia grafica è realizzata in JavaFX con layout dichiarativo FXML. La schermata principale utilizza un `TabPane` in cui le schede sono filtrate a runtime in base al ruolo dell'utente: Ristoranti è visibile a tutti, Preferiti e Le mie recensioni ai clienti, I miei ristoranti ai gestori. La navigazione è centralizzata nella classe `Navigazione` , responsabile del cambio di scena e dell'apertura dei dialoghi. I campi città si basano su `AutoCompletamento` : l'elenco viene richiesto al server una sola volta e successivamente filtrato in locale durante la digitazione, riducendo il rischio di errori di inserimento.

6. Progettazione della base di dati

6.1 Analisi dei requisiti ristrutturata

Dalle specifiche emergono i seguenti requisiti sui dati, ristrutturati per eliminare ambiguità e ridondanze:

1. Il sistema memorizza **ristoranti** con: nome (stringa); luogo, cioè nazione, città, indirizzo (stringhe) e coordinate geografiche latitudine/longitudine (numeri decimali); fascia di prezzo espressa come prezzo medio in euro (numero decimale non negativo); disponibilità di delivery e di prenotazione online (booleani); tipo di cucina (stringa).
2. Il sistema memorizza **utenti registrati** con: nome, cognome (stringhe); indirizzo e-mail univoco (stringa, identificativo di accesso); password cifrata (stringa); data di nascita facoltativa (data); luogo del domicilio (stringa); ruolo (enumerato: cliente o gestore).
3. Un **gestore** può inserire più ristoranti; un ristorante è gestito da al più un gestore (i ristoranti importati dal dataset non hanno un gestore e possono essere presi in carico da un gestore registrato).
4. Un **cliente** può recensire un ristorante *al più una volta*; la recensione ha il numero di stelle (numero intero, 1–5) e testo (stringa libera); il gestore del ristorante può rispondere *al più una volta* a ciascuna recensione (risposta come stringa libera facoltativa).
5. Un cliente mantiene una lista di ristoranti **preferiti** (senza duplicati).

6.2 Schema ER concettuale

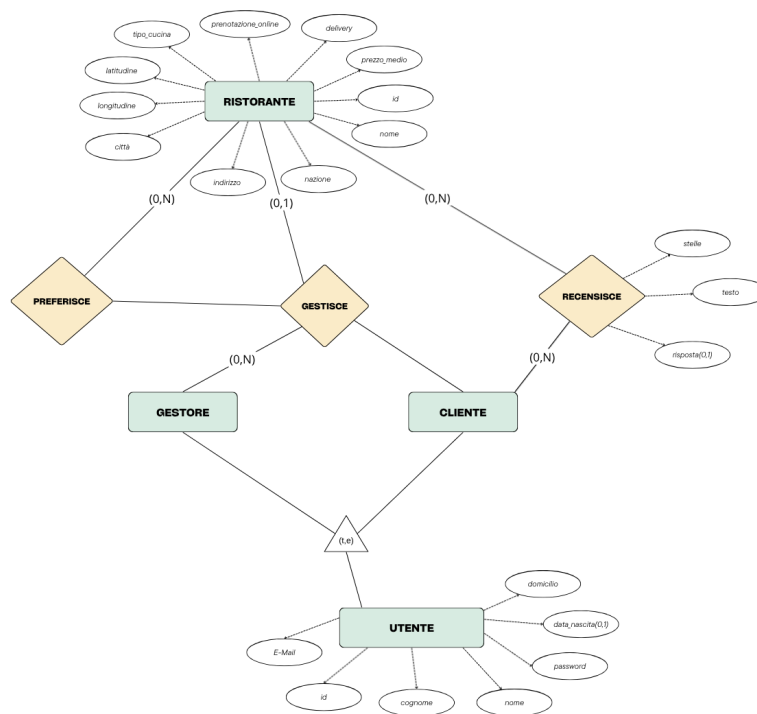


Figura 3 — Schema ER concettuale

Scelte di modellazione. La progettazione ha seguito una strategia mista: top-down per le entità principali, bottom-up per gli attributi ricavati dal dataset. CLIENTE e GESTORE sono figlie di una **generalizzazione totale ed esclusiva** di UTENTE: ogni utente registrato è esattamente uno dei due. RECENSISCE è un'associazione multi-a-molti *con attributi* (stelle, testo, risposta): la coppia (cliente, ristorante) la identifica, il che esprime naturalmente il vincolo «una recensione per utente per ristorante». PREFERISCE è multi-a-molti senza attributi. GESTISCE lega il gestore ai suoi ristoranti con cardinalità (0,1) lato ristorante.

Vincoli non esprimibili nello schema (linguaggio naturale):

- stelle è un intero compreso tra 1 e 5;
- la risposta di una recensione può essere scritta solo dal gestore che gestisce il ristorante recensito, e al più una volta;
- solo utenti con ruolo cliente possono recensire e avere preferiti; solo utenti con ruolo gestore possono gestire ristoranti;
- il prezzo medio è non negativo; la password è memorizzata solo cifrata.

6.3 Ristrutturazione dello schema ER

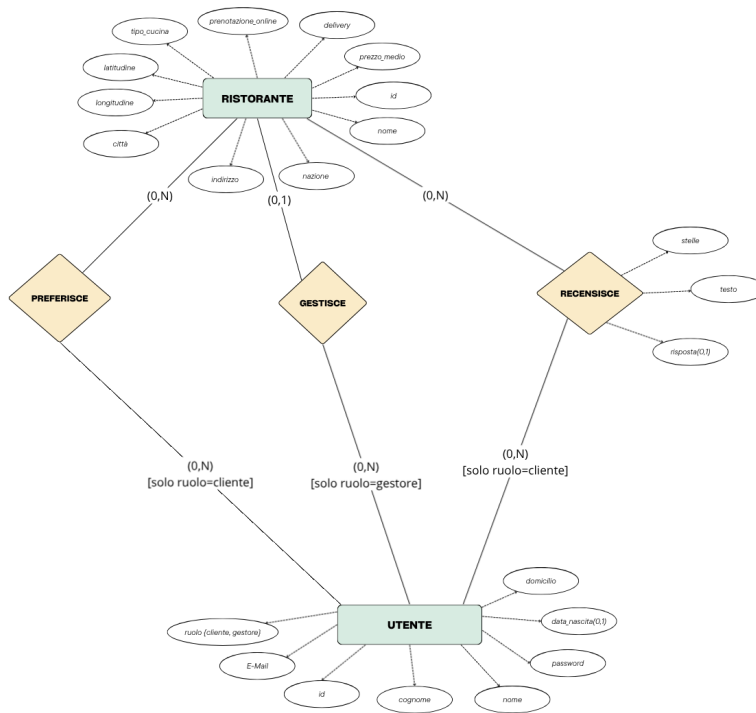


Figura 4 — Schema ER ristrutturato

Motivazioni. L'unica vera ristrutturazione riguarda la generalizzazione UTENTE/CLIENTE/GESTORE, **collassata verso il padre** aggiungendo l'attributo discriminante `ruolo`: i figli non hanno attributi propri, e l'unica differenza tra loro sta in quali associazioni li riguardano (`RECENSISCE` / `PREFERISCE` per il cliente, `GESTISCE` per il gestore), non nella struttura dei dati. Due tabelle separate avrebbero quindi solo duplicato lo schema, complicando login e registrazione (che dovrebbero cercare su due tabelle invece di una). Il vincolo di partecipazione per ruolo diventa un vincolo espresso in linguaggio naturale, fatto rispettare dal server a ogni operazione. Non vi sono attributi multivalore; l'unico attributo potenzialmente composto è `indirizzo` (via, numero civico), lasciato atomico perché l'applicazione non ha mai bisogno di interrogarne le singole parti — la scomposizione geografica rilevante (nazione, città) è già un attributo a sé. Gli attributi derivabili (media stelle, numero recensioni) *non* vengono memorizzati ma calcolati dalle query, evitando anomalie di aggiornamento.

6.4 Traduzione nello schema relazionale

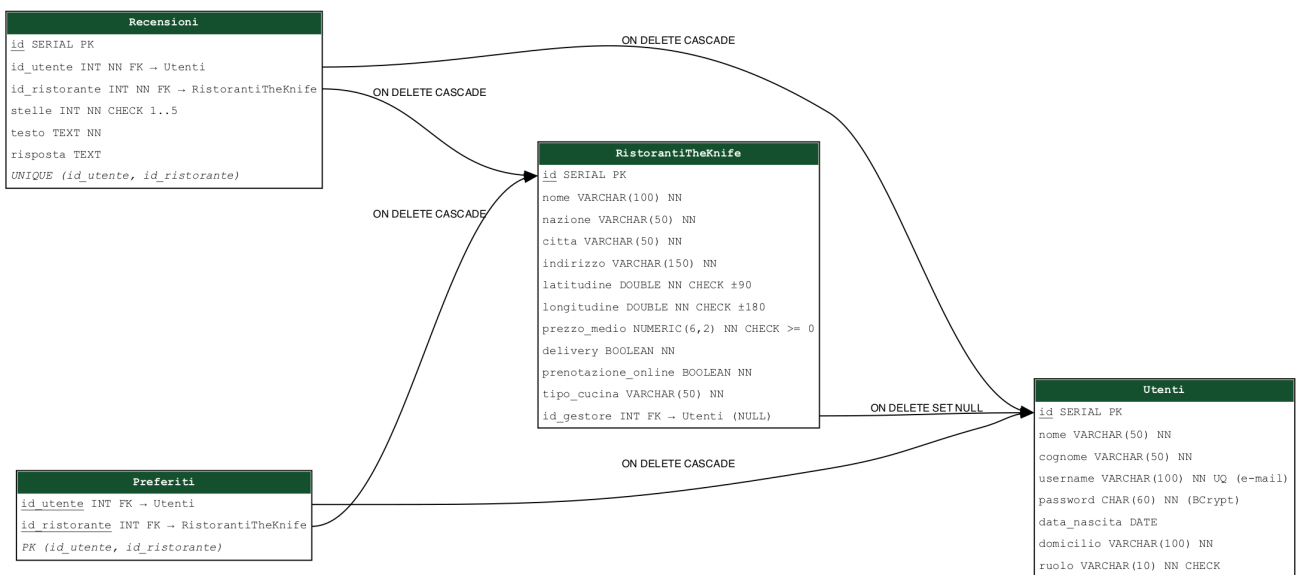


Figura 5 — Schema relazionale con vincoli di integrità referenziale

Traduzione standard dell'ER ristrutturato:

- **Utenti**(id, nome, cognome, username, password, data_nascita*, domicilio, ruolo) — da UTENTE; la colonna `username` contiene l'indirizzo e-mail ed è UNIQUE; CHECK sul ruolo.
- **RistorantiTheKnife**(id, nome, nazione, citta, indirizzo, latitudine, longitudine, prezzo_medio, delivery, prenotazione_online, tipo_cucina, id_gestore*) — da RISTORANTE; GESTISCE, avendo cardinalità massima 1 lato ristorante, è assorbita come chiave esterna `id_gestore` (NULL per i ristoranti del dataset), con `ON DELETE SET NULL` per non perdere i ristoranti se il gestore viene rimosso.
- **Recensioni**(id, id_utente, id_ristorante, stelle, testo, risposta*) — da RECENSISCE; oltre alla chiave surrogata, il vincolo `UNIQUE (id_utente, id_ristorante)` preserva l'identificazione originale della relazione; CHECK stelle 1–5; `ON DELETE CASCADE` su entrambe le FK (una recensione non ha senso senza autore o ristorante).
- **Preferiti**(id_utente, id_ristorante) — da PREFERISCE; chiave primaria composta dalle due FK, `ON DELETE CASCADE`.

Indici a supporto delle ricerche: `idx_ristoranti_citta`, `idx_ristoranti_cucina`, `idx_recensioni_ristorante`.

6.5 Script SQL e query a supporto dei servizi

Gli script versionati in `src/main/sql/`:

Script	Funzione
--------	----------

<code>schema.sql</code>	DDL: creazione del database <code>dbtk</code> (se assente, tramite i meta-comandi <code>psql \gexec</code> e <code>\connect</code> , poiché <code>CREATE DATABASE</code> non supporta <code>IF NOT EXISTS</code>) e delle quattro tabelle con tutti i vincoli (PK, FK, UNIQUE, CHECK, NOT NULL) e gli indici. Idempotente: rieseguirlo è innocuo.
<code>popola.sql</code>	Import del dataset <code>michelin_my_maps.csv</code> : tabella di staging + <code>\copy</code> , quindi <code>INSERT...SELECT</code> con le trasformazioni (split di «città, nazione», mappatura dei simboli di prezzo in prezzo medio, prenotazione online dedotta dalla presenza del sito web), infine <code>DROP</code> dello staging.

Le query dei servizi usano esclusivamente `PreparedStatement` . Le principali:

```
-- cercaRistorante(): ricerca per distanza con filtri combinati e statistiche
SELECT r.*, COALESCE(AVG(rec.stelle), 0) AS media_stelle,
       COUNT(rec.id) AS numero_recensioni,
       2*6371*ASIN(SQRT( POWER(SIN(RADIANS(r.latitudine - ?)/2), 2)
                     + COS(RADIANS(?)) * COS(RADIANS(r.latitudine))
                     * POWER(SIN(RADIANS(r.longitudine - ?)/2), 2) )) AS distanza_km
FROM RistorantiTheKnife r
LEFT JOIN Recensioni rec ON rec.id_ristorante = r.id
WHERE ...filtri opzionali (cucina, prezzo, delivery, prenotazione)...
GROUP BY r.id
HAVING distanza <= ? [AND AVG(rec.stelle) >= ?]
ORDER BY distanza_km
-- oppure: r.tipo_cucina / r.prezzo_medio [DESC] / media_stelle DESC
--          -- sempre seguito da , distanza_km
```

```
-- rivendicaRistorante(): presa in carico atomica di un ristorante senza gestore
UPDATE RistorantiTheKnife SET id_gestore = ?
WHERE id = ? AND id_gestore IS NULL;
```

```
-- rispostaRecensione(): risposta unica, atomica, solo del proprietario
UPDATE Recensioni rec SET risposta = ?
FROM RistorantiTheKnife r
WHERE rec.id = ? AND rec.risposta IS NULL
      AND r.id = rec.id_ristorante AND r.id_gestore = ?;
```

```
-- visualizzaRiepilogo(): valutazione media e numero recensioni del gestore
SELECT r.*, COALESCE(AVG(rec.stelle), 0) AS media_stelle,
       COUNT(rec.id) AS numero_recensioni
FROM RistorantiTheKnife r
LEFT JOIN Recensioni rec ON rec.id_ristorante = r.id
WHERE r.id_gestore = ?
GROUP BY r.id ORDER BY r.nome;
```

7. Algoritmi e strutture dati

7.1 Ricerca per distanza (Haversine + baricentro)

Le specifiche richiedono l'elenco dei ristoranti *vicini* a un luogo (testo libero, senza coordinate).

L'algoritmo adottato:

1. **Geocoding interno:** il punto di riferimento del luogo è il *baricentro* (media di latitudine e longitudine) dei ristoranti della città indicata: le coordinate arrivano dal dataset stesso, senza bisogno di un servizio di geocoding esterno;
2. **Distanza ortodromica** di ogni ristorante dal riferimento con la *formula di Haversine* (raggio terrestre 6371 km), calcolata direttamente in SQL così che filtro (`<= raggio massimo` , predefinito 30 km) e ordinamento avvengano nel DBMS in un'unica passata, senza trasferire 17.000 righe al server applicativo. Il punto di partenza è già la città indicata, quindi il minimo è sempre 0.
3. **Ordinamento selezionato dall'utente:** l'utente può ordinare i risultati per distanza (default), tipo di cucina, prezzo o valutazione. Il nome della colonna da usare nell' `ORDER BY` non può essere legato con un parametro `?` di un `PreparedStatement` , perché questo parametrizza solo valori, non nomi di colonna. Per questo `RistoranteDAO.ordinePer()` traduce il criterio scelto in una delle clausole `ORDER BY` scritte nel codice: il client seleziona solo tra opzioni predefinite, non invia mai testo SQL. In ogni caso la distanza resta il criterio finale: a parità di cucina, prezzo o valutazione, la priorità è il ristorante più vicino.

L'ordinamento per valutazione riusa un valore già calcolato dalla query: `media_stelle` è `COALESCE(AVG(rec.stelle), 0)` , quindi un ristorante senza recensioni vale sempre `0` , mentre uno con almeno una recensione vale sempre `1` o più. Basta un semplice `DESC` per mettere prima tutti i ristoranti con recensioni (dal voto più alto), e dopo quelli senza, già ordinati per distanza grazie al criterio finale. Se nessun risultato ha recensioni, tutti valgono `0` : il criterio non fa differenza, e l'ordinamento finale coincide con la semplice distanza.

7.2 Coerenza dei dati

Media stelle e numero recensioni non sono colonne salvate nel database: si calcolano a ogni query con `AVG` e `COUNT` su un `LEFT JOIN` . Così restano sempre aggiornate, senza bisogno di sincronizzarle ogni volta che arriva una nuova recensione.

7.3 Autocompletamento delle città

L'elenco delle città distinte (~5.800 stringhe) viene richiesto al server una sola volta per sessione e tenuto nella cache del client; a ogni tasto l'elenco viene filtrato con un semplice ciclo che confronta le stringhe in minuscolo e si ferma a 50 suggerimenti. Ogni tasto premuto scansiona tutte le n città: complessità $O(n)$ per digitazione.

7.4 Strutture dati del protocollo

La mappa parametri di `Request` (`HashMap` chiave→Object) permette di aggiungere nuovi parametri senza creare nuove classi. I risultati restituiti dal server sono liste (`ArrayList`) serializzate insieme alla `Response`. Sul client, le `ListView` mostrano questi risultati con una cell factory dedicata per tipo: una per i ristoranti (che aggiunge distanza e valutazione), una per le recensioni (stelle, testo, risposta).

8. Sicurezza

- **Password cifrate con BCrypt** (libreria `org.mindrot:jbcrypt`). Prima della cifratura, `Utente.passwordValida()` (package `theknife.common`, stessa logica di `emailValida()`) verifica i requisiti minimi — almeno 8 caratteri, con almeno una lettera e una cifra — sia lato client sia lato server. In registrazione, `BCrypt.hashpw` genera un hash con salt individuale, salvato in una colonna `CHAR(60)`. In login, `BCrypt.checkpw` confronta la password ricevuta con l'hash, senza mai decifrarlo. La password in chiaro non viene mai salvata, e l'hash non lascia mai il server: prima di inviare i dati al client, il campo password viene azzerato.
- **SQL**: tutte le query usano `PreparedStatement` con parametri posizionali; le parti dinamiche della query di ricerca aggiungono solo frammenti SQL costanti, mai input dell'utente.
- **Autorizzazione lato server**: identità e ruolo dell'utente risiedono nella sessione del `ClientHandler`, non in dati inviati dal client. Le query di modifica includono sempre questa identità nella clausola WHERE (es. `id_utente = ?` per modificare una recensione). Un client contraffatto non può quindi agire su dati altrui: anche inviando un id diverso, la query filtra comunque per l'utente realmente autenticato.
- **Recensioni anonime**: il DAO non copia mai l'identificativo dell'autore nelle recensioni inviate ai client, come richiesto dalle specifiche.

9. Controllo degli errori e gestione delle eccezioni

- **Server**: il dispatcher (smistatore) racchiude ogni operazione in un try/catch che converte qualunque eccezione in `Response.errore(...)` — il client riceve sempre una risposta e la connessione sopravvive all'errore. Le violazioni di vincolo attese (`SQLState 23505`) sono tradotte in messaggi specifici. La disconnessione del client (`EOFException`) chiude la sessione in modo pulito; socket e connessioni JDBC sono gestiti con try-with-resources. Anche l'avvio del server verifica subito la connessione al database e termina con un messaggio chiaro se i parametri sono errati.
- **Validazione doppia**: il client controlla campi obbligatori, formati numerici e conferma password, per dare un feedback immediato. Il server ripete comunque tutti i controlli (stelle 1–5, testo non vuoto, prezzo non negativo, coordinate nei range geografici validi, anche via CHECK nello schema), perché è l'unico responsabile della correttezza dei dati — il client serve

solo per l'usabilità. Per un nuovo ristorante, latitudine e longitudine sono obbligatorie: non vengono mai derivate dal baricentro della città, altrimenti ristoranti diversi nella stessa città risulterebbero sovrapposti nello stesso punto, con identica distanza da qualunque riferimento. Se la città è già nota, le coordinate indicate devono comunque distare al massimo 60 km dal baricentro: così un valore sbagliato non falsa la ricerca per distanza per tutta la città.

- **Client:** un errore di comunicazione con il server (es. connessione rifiutata, timeout) diventa una `Response` di errore, mostrata all'utente in un'etichetta o in un `Alert`.

10. Compilazione, packaging e struttura della consegna

Build Maven (`pom.xml`): Java release 21, dipendenze OpenJFX 21 (controls, fxml), driver PostgreSQL 42.7, jBCrypt 0.4.

- `mvn clean package` — compila e produce, tramite *maven-shade-plugin* con due esecuzioni, i fat-jar `bin/serverTK.jar` (main: `theknife.server.ServerTK`) e `bin/clientTK.jar` (main: `theknife.client.Launcher`). Launcher esiste perché JavaFX richiederebbe un `module-path` per avviare un' `Application` direttamente da un fat-jar: una classe separata, che non estende `Application`, aggira il vincolo;
- `mvn javadoc:javadoc` — genera la javadoc in `doc/apidocs` ;
- `mvn javafx:run` — esecuzione rapida del client in sviluppo.

Struttura del repository (come da specifiche di consegna):

```
theknife/
  autori.txt          dati dell'autore
  README.txt          istruzioni di installazione e compilazione
  pom.xml             build Maven
  bin/                serverTK.jar, clientTK.jar
  src/main/java/      sorgenti (package theknife.*)
  src/main/resources/ FXML delle schermate
  src/main/sql/       schema.sql, popola.sql
  lib/                (librerie gestite da Maven)
  doc/                manuale utente e tecnico (PDF), diagrammi/ (ER, UML),
                      screenshots/, apidocs/ (javadoc)
```

Ogni file `.java` riporta in intestazione nome, cognome, matricola e sede dell'autore; tutte le classi e i metodi pubblici sono documentati in formato javadoc.